

Browser-Based 3D for a Box-Net and Artwork App

Executive summary

For a web app that first builds a precise 2D box net, then lets the user place artwork, and finally offers a 3D folded preview, the most robust architecture is **client-first, millimeter-native, and 2D-authoritative**. In practice, that means storing all structural and artwork placement data in **millimeters** in a single model, deriving the 3D preview from that model, and treating export as a deterministic rendering of the same source of truth. This avoids the classic failure mode where the 2D editor, UV layout, 3D preview, and export pipeline drift apart. Three.js is particularly strong here because it gives direct control over `BufferGeometry`, UVs, textures, raycasting, controls, and WebGL/WebGPU rendering, while browser-native storage and offline APIs support local processing and privacy-preserving workflows. PDF.js and pdf-lib complement that stack well: PDF.js is excellent for parsing and rasterizing incoming PDFs, and pdf-lib is strong for building/exporting PDFs, embedding pages, clipping, and drawing vector dielines. ¹

The core implementation insight is simple but important: **do not unwrap a folded mesh after the fact**. Instead, compute each panel's UVs directly from the flat net coordinates. If the artwork is conceptually “a single image placed under the dieline,” then each panel's UVs are just a mapping from its flat net-space rectangle into that single artwork rectangle. Once those UVs are assigned, the 3D preview can fold panels in space without changing texture coordinates. The preview becomes a pure transform problem, not a second artwork-placement problem. ²

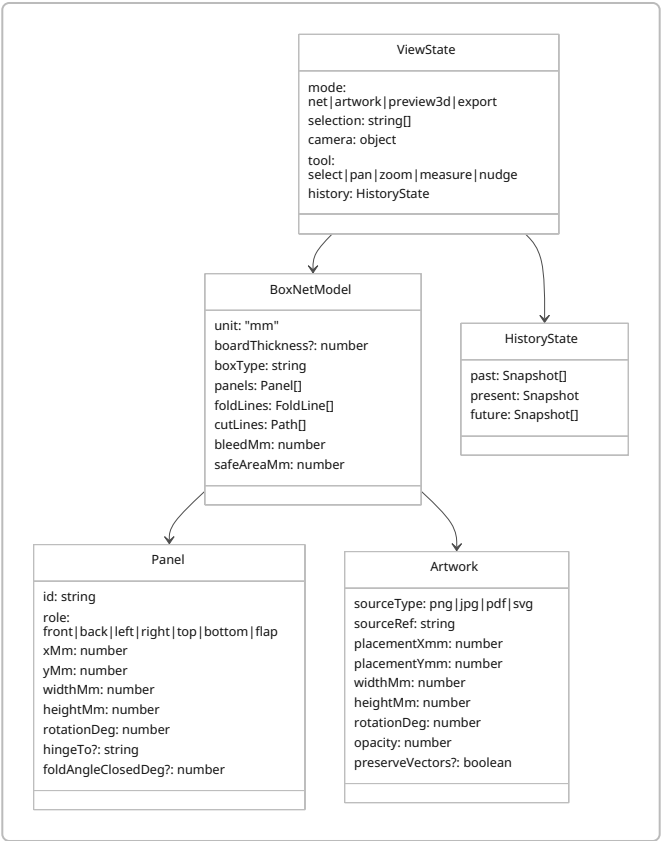
A practical recommended stack for this app, assuming no server constraints are specified, is: **TypeScript**, optional **React**, **SVG** for structural/dieline editing, **Konva** or **Fabric** only if you want higher-level 2D transforms and object editing, **Three.js** for 3D folding/preview, **PDF.js** for PDF import/view/rasterization, **pdf-lib** for export, **IndexedDB** for local project storage, and a **Service Worker** for offline readiness. If React is already central to the product, React Three Fiber is a good ergonomics layer over Three.js, but it comes with documented performance pitfalls when high-frequency scene updates are pushed through React state rather than imperative loops. ³

For this class of application, **precise placement should remain authoritative in 2D**, not in 3D. Users may inspect and validate the result in 3D, but the canonical edit surface should stay flat and measured. If you still want “precise mm-level positioning in 3D,” the best UX is to let users pick a panel in 3D and edit **panel-local coordinates** shown in millimeters, rather than direct world-space transforms. That keeps the data model aligned with print/export requirements. This recommendation is an inference from the coordinate-system and transform behavior of the underlying browser and 3D APIs, and from the fact that on-screen physical CSS units are not guaranteed to match real-world measurements on displays. ⁴

Priority sources for this topic include the Three.js docs on `BufferGeometry`, `Object3D`, `WebGLRenderer`, and `WebGPURenderer`; Mozilla's PDF.js docs and examples; pdf-lib's API docs; Redux's undo-history guidance; and MDN's storage/offline documentation. ⁵

Recommended architecture and stack options

The best architecture separates the app into four layers: a **structural model** in millimeters, an **artwork placement model** also in millimeters, a **view state** for editor/preview UI, and **render/export adapters** that translate the canonical model into SVG, canvas, Three.js meshes, and PDF output. Undo/redo should wrap the canonical state, not the canvas/scene internals. Redux's official undo-history guidance is directly relevant here because it formalizes the `past / present / future` pattern for immutable state, while Konva's undo/redo guidance similarly recommends storing history as plain state snapshots rather than scraping state back out of the canvas. 6



Framework and engine comparison

Stack option	Where it fits best	Main strengths	Main trade-offs	Sources
TypeScript + SVG + Three.js + PDF.js + pdf-lib	Best overall fit for a packaging editor with print/ export precision	SVG is excellent for vector dielines and clipping; Three.js gives low-level control over geometry, UVs, textures, controls, and WebGL/WebGPU; PDF.js and pdf-lib cover import/ export well	More custom integration work; you build more editor behavior yourself	7

Stack option	Where it fits best	Main strengths	Main trade-offs	Sources
TypeScript + React + SVG/ Konva + React Three Fiber + Drei + PDF.js + pdf-lib	Best if the rest of the product is already React-heavy	Excellent React ergonomics; mature ecosystem around controls and loaders; good componentization	pmndrs docs explicitly warn about performance pitfalls and scaling concerns if fast-changing 3D state is pushed reactively	8
TypeScript + Fabric.js + Three.js + PDF.js + pdf-lib	Good if users need richer 2D object editing, serialization, or SVG export from the artwork editor	Fabric supports selections, serialization, export to JSON/SVG/image, and clip paths	Fabric is canvas-centric; structural/ vector correctness for print still needs careful handling; integrating Fabric state with Three.js and export adds complexity	9
Babylon.js + SVG/HTML 2D + PDF.js + pdf-lib	Good alternative if your team prefers a more batteries-included engine	Official WebGPU support, thin instances, KTX2 support, engine-level optimization tooling	Less commonly chosen for print-style 2D editors; custom dieline/UV/ export workflow is still bespoke	10
PlayCanvas + HTML/SVG 2D + PDF.js + pdf-lib	Best if the app may evolve toward a more content-pipeline-driven 3D product	WebGL/WebGPU support, asset tooling, profiling docs, browser support guidance	More opinionated engine/editor ecosystem; less natural for print-precise 2D authoring than SVG-first approaches	11

For this project specifically, the strongest recommendation is **SVG-first 2D + Three.js 3D**. SVG gives you crisp structural lines, accessible DOM-adjacent UI, straightforward clip paths, and a cleaner bridge to vector export. If you want ready-made transform handles and selection boxes for artwork placement, add Konva or Fabric only around the artwork layer, not as the canonical structural model. Three.js remains the best fit for generating the folded preview from custom panel geometry because its `BufferGeometry`, `BufferAttribute`, controls, raycasting, and texture APIs are all explicitly designed for this sort of low-level scene construction. 12

If no server requirement is specified, prefer **local/offline processing by default**. Browser support is strong for offline-first caching with Service Workers, significant local storage with IndexedDB, and user-approved read/write integration through the File System Access API. This approach reduces privacy risk for packaging artwork and avoids upload latency. A server can still be added later for heavy PDF/vector conversions, color-management workflows, or collaboration. 13

Priority sources for this topic include Redux's official undo-history docs, Konva's transformer and undo/redo guides, Fabric's core concepts and clipping docs, Three.js' renderer and geometry docs, and MDN's offline/storage docs. ¹⁴

Geometry, UV mapping, artwork placement, and masking

A custom box app should model panels in **flat-net space first**. Each panel is a rectangle or flap path with a known position and orientation in millimeters. From that 2D representation, you create both the dieline view and the 3D preview. Three.js `BufferGeometry` explicitly supports vertex positions, face indices, normals, colors, UVs, and custom attributes, which makes it well-suited to a panel-by-panel procedural construction pipeline. ¹⁵

One important unspecified detail is **box style**. A “box from W×H×D” is not fully defined until you choose a structural schema: for example, a simple six-panel rigid fold box, a reverse-tuck carton, a mailer, or a slotted box. Those styles differ mainly in flap topology and hinge relationships, not in the core principle of panel construction. The implementation pattern below therefore uses a **minimal six-face rectangular box** and treats flaps/glue tabs as additional panel variants.

Building box geometry from W×H×D

A reliable pattern is to store each panel twice: once as a **flat net rectangle** and once as a **3D scene node** with a hinge pivot. In the flat model, all coordinates are in millimeters. In the 3D model, those same millimeter values can be used directly as world units, which makes fold math and panel-local measurements intuitive.

```
type PanelRole = 'front' | 'back' | 'left' | 'right' | 'top' | 'bottom';

interface Panel2D {
  id: string;
  role: PanelRole;
  xMm: number;      // flat-net origin
  yMm: number;
  wMm: number;
  hMm: number;
  hingeTo?: string; // parent panel in fold graph
  hingeEdge?: 'left' | 'right' | 'top' | 'bottom';
  foldClosedDeg?: number; // target angle when fully folded
}

/**
 * Minimal six-panel box net.
 * Flap geometry is intentionally omitted because box style is unspecified.
 */
function buildRectangularBoxNet(W: number, H: number, D: number): Panel2D[] {
  return [
    { id: 'front', role: 'front', xMm: D, yMm: D, wMm: W, hMm: H },
```

```

    { id: 'left',   role: 'left',   xMm: 0,     yMm: D, wMm: D, hMm: H,
      hingeTo: 'front', hingeEdge: 'left', foldClosedDeg: 90 },
    { id: 'right',  role: 'right',  xMm: D + W, yMm: D, wMm: D, hMm: H,
      hingeTo: 'front', hingeEdge: 'right', foldClosedDeg: -90 },
    { id: 'back',   role: 'back',   xMm: D + W + D, yMm: D, wMm: W, hMm: H,
      hingeTo: 'right', hingeEdge: 'right', foldClosedDeg: -90 },
    { id: 'top',    role: 'top',    xMm: D,      yMm: 0, wMm: W, hMm: D,
      hingeTo: 'front', hingeEdge: 'top',   foldClosedDeg: -90 },
    { id: 'bottom', role: 'bottom', xMm: D,      yMm: D + H, wMm: W, hMm: D,
      hingeTo: 'front', hingeEdge: 'bottom', foldClosedDeg: 90 },
  ];
}

```

This pattern works because the 2D net is the authoritative layout, while the 3D scene graph simply reuses those dimensions and relationships. `Object3D` exists precisely to manipulate objects in 3D space, and a hierarchy of hinge pivots makes fold animation straightforward. ¹⁶

Generating UVs for each panel from the flat artwork placement

If the user places **one single artwork image** under the whole net, per-panel UVs can be computed directly from net coordinates. That is the cleanest possible mapping, because you do not need an atlas packer or an unwrap step. Every panel vertex already has a position in flat-net space; just normalize those coordinates into the artwork rectangle.

```

interface ArtworkPlacement {
  xMm: number;
  yMm: number;
  wMm: number;
  hMm: number;
}

function uvFromNetPoint(netX: number, netY: number, art: ArtworkPlacement) {
  const u = (netX - art.xMm) / art.wMm;
  const v = 1 - (netY - art.yMm) /
    art.hMm; // top-left 2D editor to bottom-left texture convention if needed
  return { u, v };
}

function buildPanelGeometry(panel: Panel2D, art: ArtworkPlacement) {
  const x0 = panel.xMm;
  const y0 = panel.yMm;
  const x1 = panel.xMm + panel.wMm;
  const y1 = panel.yMm + panel.hMm;

  // panel-local plane geometry centered on origin in 3D
  const positions = new Float32Array([

```

```

    -panel.wMm / 2, -panel.hMm / 2, 0,
    panel.wMm / 2, -panel.hMm / 2, 0,
    panel.wMm / 2, panel.hMm / 2, 0,
    -panel.wMm / 2, panel.hMm / 2, 0,
  ]);

  // UVs are derived from FLAT net-space corners
  const uv00 = uvFromNetPoint(x0, y1, art);
  const uv10 = uvFromNetPoint(x1, y1, art);
  const uv11 = uvFromNetPoint(x1, y0, art);
  const uv01 = uvFromNetPoint(x0, y0, art);

  const uvs = new Float32Array([
    uv00.u, uv00.v,
    uv10.u, uv10.v,
    uv11.u, uv11.v,
    uv01.u, uv01.v,
  ]);

  const indices = [0, 1, 2, 0, 2, 3];

  return { positions, uvs, indices };
}

```

This approach aligns directly with how `BufferGeometry` and `BufferAttribute` are designed to carry positions and UVs into the GPU. It is also especially stable for export, because the same panel net coordinates drive SVG clipping, canvas rasterization, and 3D texturing. ²

Mapping one artwork image under the dieline in 2D

For 2D editing, there are three sensible strategies:

Strategy	Best use	Pros	Trade-offs	Sources
One full-net image placed under SVG dieline	Best default for continuity across panel boundaries	One transform for the user; easiest mental model; vector clip paths per panel are straightforward	Large raster images can become memory-heavy	¹⁷
Canvas texture generated from artwork + overlays	Best when raster filters or compositing are required	Easy to composite images, PDFs, masks, guides	Canvas export/readback is subject to CORS tainting; default file exports carry 96dpi metadata	¹⁸

Strategy	Best use	Pros	Trade-offs	Sources
Per-panel cropped textures	Best if each panel is distinct or if memory capping is critical	Smaller textures per mesh; simplifies per-panel edits	Continuity across seams is harder; more state to manage	19
Vector-preserving 2D plus rasterized 3D texture	Best if print fidelity matters more than 3D exact vector preservation	Keep SVG/PDF vectors for export while using a fast raster texture for preview	Requires dual representation and careful synchronization	20

A lightweight SVG implementation pattern looks like this:

```
<svg id="editor" viewBox="0 0 500 400">
  <defs>
    <clipPath id="panel-front">
      <rect x="80" y="80" width="200" height="120" />
    </clipPath>
  </defs>

  <!-- Full artwork image placed once -->
  <image
    href="artwork.png"
    x="40" y="40"
    width="320" height="220"
    clip-path="url(#panel-front)" />

  <!-- Dieline drawn above -->
  <rect x="80" y="80" width="200" height="120"
    fill="none" stroke="black" stroke-width="0.3" />
</svg>
```

SVG's `clipPath` is a very strong fit for panel masking because it precisely restricts the region to which paint is applied. If you use Fabric instead, its `clipPath` support and absolute clip paths provide a comparable mental model inside a canvas-based editor. 21

Image normalization and masking pipeline

For uploads, a modern and performant raster pipeline is: decode the source with `createImageBitmap()`, normalize orientation and optional resize in a worker when needed, composite into an `OffscreenCanvas`, and pass the result as an `ImageBitmap` or canvas-backed texture to the 2D or 3D layer. `ImageBitmap` is explicitly described by MDN as an efficient pathway for preparing textures for WebGL, and `OffscreenCanvas` can produce `ImageBitmap` outputs without an extra transfer copy. Remember to call `ImageBitmap.close()` when a bitmap is retired. 22

```
// Worker-side pseudocode
self.onmessage = async (e) => {
  const blob = e.data as Blob;

  const bmp = await createImageBitmap(blob, {
    imageOrientation: 'from-image',
    colorSpaceConversion: 'default',
    resizeWidth: 4096,    // cap only if policy requires
    resizeHeight: 4096,
    resizeQuality: 'high',
  });

  const canvas = new OffscreenCanvas(bmp.width, bmp.height);
  const ctx = canvas.getContext('2d')!;
  ctx.drawImage(bmp, 0, 0);

  const out = canvas.transferToImageBitmap();
  bmp.close();

  // Transfer to main thread for texture upload or editor display
  postMessage(out, [out]);
};
```

Priority sources for this topic include the Three.js docs on `CanvasTexture`, `BufferGeometry`, and `BufferAttribute`; MDN's `ImageBitmap`, `OffscreenCanvas`, `createImageBitmap`, and SVG clipping docs; and Fabric/Konva documentation for object transforms and clipping. ²³

3D preview, fold animation, interaction patterns, and performance

The 3D preview should be a **derived presentation layer**. Each panel is a plane mesh; each hinge is an `Object3D` pivot; and folding is achieved by rotating pivots, not by deforming the mesh. This is simpler, easier to debug, and more export-stable than trying to build a skinned mesh for simple cartons. Three.js' `Object3D` hierarchy and `AnimationMixer` are the relevant primitives here. ¹⁶

Fold animation pattern

```
/**
 * Create a hinge pivot for one panel.
 * The panel mesh sits offset from the pivot so rotating the pivot folds around
 * the edge.
 */
function makePanelNode(panel: Panel2D, geometry: THREE.BufferGeometry, texture:
THREE.Texture) {
  const pivot = new THREE.Object3D();
  const mesh = new THREE.Mesh(
```



```

    geometry,
    new THREE.MeshStandardMaterial({ map: texture })
  );

  // panel mesh is centered; offset it from the hinge pivot
  switch (panel.hingeEdge) {
    case 'left':
      mesh.position.x = panel.wMm / 2;
      break;
    case 'right':
      mesh.position.x = -panel.wMm / 2;
      break;
    case 'top':
      mesh.position.y = -panel.hMm / 2;
      break;
    case 'bottom':
      mesh.position.y = panel.hMm / 2;
      break;
  }

  pivot.add(mesh);
  return { pivot, mesh };
}

function applyFold(panel: Panel2D, pivot: THREE.Object3D, t: number) {
  const target = THREE.MathUtils.degToRad(panel.foldClosedDeg ?? 0);
  const angle = THREE.MathUtils.lerp(0, target, t);

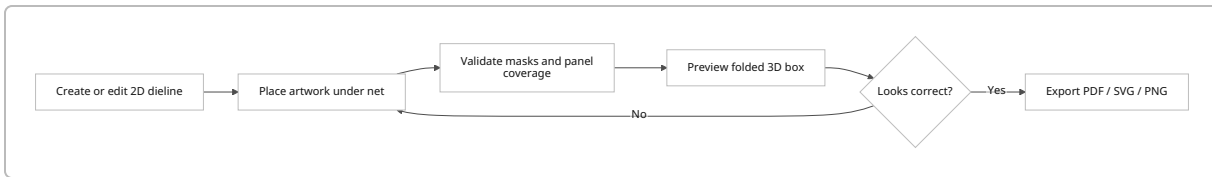
  if (panel.hingeEdge === 'left' || panel.hingeEdge === 'right') {
    pivot.rotation.y = angle;
  } else {
    pivot.rotation.x = angle;
  }
}

```

This “rigid panel + hinge pivot” pattern closely matches the real mechanics of a folded carton, and it keeps texture coordinates intact because only transforms are changing. If you want timeline-based easing, `AnimationMixer` can drive these rotations, but for many packaging apps a deterministic `t` parameter controlled by a slider is even better UX. ¹⁶

Recommended UX between 2D and 3D

The most reliable UX is a **mode-based flow** in which the user edits structure in 2D, positions artwork in 2D, validates in 3D, then exports. The 3D preview should be optimized for **inspection**, not for being a second full editor. Orbit/pan/zoom belong in preview; mm-precise nudge/edit belongs in 2D. Three.js `OrbitControls` already maps well to desktop and touch interactions, while `Raycaster` and `TransformControls` are appropriate if you still want panel selection or limited manipulation in 3D. ²⁴



A good interaction pattern is to keep a **persistent selection identity** across modes. If the user selects the “front panel” in 2D, that same panel should highlight in 3D. In preview mode, clicking a panel with a `Raycaster` should focus it, show panel-local dimensions and artwork coverage, and optionally open a numeric inspector for panel-local offsets. This works far better than letting users drag arbitrary world-space gizmos for print placement. ²⁵

For **touch**, rely on Pointer Events and set `touch-action` deliberately. MDN notes that browsers otherwise reserve panning and pinch gestures for themselves, causing `pointercancel` when the browser takes over. For this app, a common rule set is: the canvas/editor surface gets `touch-action: none` only while actively manipulating artwork or the 3D camera; the rest of the UI stays scrollable and native. MDN’s pinch-zoom examples are useful reference material for custom two-pointer zoom behavior. ²⁶

Precise mm-level positioning in 3D

If you must support precise placement in 3D, the most robust pattern is:

1. Use `Raycaster` to pick the panel.
2. Transform the hit point into **panel-local coordinates**.
3. Display those coordinates in millimeters relative to the panel origin.
4. Let users adjust values numerically or with constrained nudge buttons.
5. Write the changes back to the shared 2D artwork-placement model.

This is an architectural recommendation rather than a documented API recipe, but it is strongly supported by the fact that `Raycaster` gives panel hits, `TransformControls` provides a DCC-like interaction model, and the canonical data should remain immutable and undoable to preserve time-travel/history semantics. ²⁷

Performance, memory, and GPU handling

Although a single-box preview is not a “heavy 3D scene” by game standards, browser graphics pipelines still need careful resource management. The official MDN WebGL best-practices guide recommends aggressively managing resource lifetime, estimating a VRAM budget, batching draw calls, using mipmaps, considering compressed textures, and sometimes rendering to a smaller back buffer for speed. Three.js also expects you to dispose materials and textures when they are no longer needed, and its `Texture` docs warn that once a texture has been used, you cannot safely mutate its dimensions/format/type in place; instead you should dispose and recreate it. ²⁸

A concrete performance policy for this app could be:

- Keep the **2D structural editor vector-first** and light.

- For the 3D preview, cap render resolution on weaker devices by reducing the drawing buffer or pixel ratio.
- Use mipmaps for large artwork textures when the box can zoom far away, and cache a lower-resolution preview texture for fast interaction.
- Call `dispose()` on old textures/materials when users replace artwork repeatedly.
- If you later support “sheet views” or multiple copies of the same box, use `InstancedMesh` to reduce draw calls. ²⁹

If you anticipate heavier 3D content later, KTX2/Basis textures are worth knowing. Khronos and Three.js both document KTX2/Basis as a way to keep textures compressed in GPU memory, reducing upload cost and memory footprint, although for a **single user-uploaded artwork image** they are usually unnecessary in the first versions of this app. They become more relevant once you add richer materials, lighting environments, or reusable decorative assets. ³⁰

Priority sources for this topic include Three.js’ docs for `Object3D`, `AnimationMixer`, `OrbitControls`, `TransformControls`, `Raycaster`, `Texture`, `Material`, and `InstancedMesh`, together with MDN’s WebGL best-practices and Pointer Events documentation. ³¹

Coordinate systems, accurate physical size, PDF/vector handling, and export

For a packaging app, the safest rule is: **store dimensions in millimeters, convert only at the boundary**. Use millimeters for all structure and artwork placement in the model, convert to CSS pixels for on-screen layout, to raster pixels for PNG/canvas exports, and to points for PDF generation.

The key formulas are:

```
const MM_PER_INCH = 25.4;
const CSS_PX_PER_INCH = 96;
const PDF_PT_PER_INCH = 72;

const mmToCssPx = (mm: number) => mm * CSS_PX_PER_INCH / MM_PER_INCH;
const mmToPt    = (mm: number) => mm * PDF_PT_PER_INCH / MM_PER_INCH;
const mmToRasterPx = (mm: number, dpi: number) => mm * dpi / MM_PER_INCH;
```

These formulas follow directly from the CSS absolute-units spec, which defines `1in = 96px`, `1pt = 1/72in`, and fixed relationships among mm, in, pt, and px. ³²

One subtle but important point: on **screens**, CSS physical units are not guaranteed to match the real world. The CSS spec explicitly notes that for screen media the anchor unit is often the pixel, and physical units may not match their physical measurements. That means your app should treat “1 mm” as **data truth**, not as “exactly 1 mm on the user’s monitor.” If true physical on-screen rulers matter, add an optional calibration step; otherwise keep all precision claims attached to export/print, not to display size. ³²

PDF introduces another coordinate-system wrinkle. PDF.js documents that a page viewport is based on a 72-DPI page space and that PDF coordinates use **bottom-left origin**, while HTML canvas uses **top-left origin**. If you rasterize or inspect incoming PDFs in the browser, you must handle this origin flip carefully. Three.js and raw WebGPU introduce their own coordinate conventions, so keeping one internal, canonical millimeter geometry model is the simplest way to avoid confusion. ³³

Export formats and what they are good for

Format	Best use	Preserves vectors	Physical-size fidelity	Main caveat	Sources
PDF	Print production, dielines, trim/bleed, mixed raster/vector	Yes, if you keep the path/page vector data	Excellent when page size, TrimBox/ BleedBox, and placements are authored correctly	Vector preservation for arbitrary imported artwork is harder than simple raster placement	³⁴
SVG	Structural/vector export, previews, downstream editing	Yes	Strong for geometry; depends on consumer for print workflow	Full print-prepress workflows are usually PDF-centric	³⁵
PNG	Quick proofs, thumbnails, 3D snapshot export	No	Only if raster dimensions are computed at a chosen DPI	Canvas export files are encoded with 96dpi metadata when supported, so physical print size should be treated carefully	³⁶

Exporting a 1:1 PDF with vector dieline and raster or vector artwork

pdf-lib supports creating pages, embedding images, embedding PDF pages, drawing lines, drawing SVG paths, setting TrimBox, and pushing low-level clipping operators. That is enough to build a credible 1:1 export pipeline for this app. ³⁷

```
import {
  PDFDocument,
  rgb,
  pushGraphicsState,
  popGraphicsState,
  moveTo,
  lineTo,
  closePath,
  clip,
} from 'pdf-lib';
```

```

async function exportNetPdf(netWmm: number, netHmm: number, panels: Panel2D[],
artworkPng: Uint8Array) {
  const pdf = await PDFDocument.create();
  const page = pdf.addPage([mmToPt(netWmm), mmToPt(netHmm)]);

  // Trim box = finished intended page dimensions
  page.setTrimBox(0, 0, mmToPt(netWmm), mmToPt(netHmm));

  // Embed one raster artwork image placed once at full net position
  const img = await pdf.embedPng(artworkPng);

  for (const p of panels) {
    const x = mmToPt(p.xMm);
    const y = mmToPt(netHmm - (p.yMm + p.hMm)); // convert top-left net to PDF
bottom-left
    const w = mmToPt(p.wMm);
    const h = mmToPt(p.hMm);

    // Clip to panel rectangle
    page.pushOperators(
      pushGraphicsState(),
      moveTo(x, y),
      lineTo(x + w, y),
      lineTo(x + w, y + h),
      lineTo(x, y + h),
      closePath(),
      clip(),
    );

    // Draw artwork once, aligned to whole net
    page.drawImage(img, {
      x: mmToPt(/* artwork.xMm */ 0),
      y: mmToPt(/* netHmm - artwork.yMm - artwork.hMm */ 0), // compute from
placement
      width: mmToPt(/* artwork.wMm */ netWmm),
      height: mmToPt(/* artwork.hMm */ netHmm),
    });

    page.pushOperators(popGraphicsState());

    // Dieline on top
    page.drawRectangle({
      x, y, width: w, height: h,
      borderColor: rgb(0, 0, 0),
      borderWidth: 0.3,
    });
  }
}

```

```
    return await pdf.save();  
  }
```

If your dieline is already represented as SVG paths, pdf-lib's documented `drawSvgPath()` API is a good high-level way to render those vector cut/fold lines. For trim/bleed workflows, pdf-lib also exposes `getTrimBox()` and `setTrimBox()`, which is useful when the finished size after trimming matters distinctly from the media size. ³⁸

Handling PDFs from users: rasterize or preserve vectors

The right answer depends on the product promise.

Rasterize with PDF.js when:

- the imported PDF is being treated like “just another artwork source,”
- users need arbitrary transforms, filters, or masking,
- fast preview/edit responsiveness matters more than preserving vector primitives,
- the file is large or inconsistent.

PDF.js' documented role is parsing/rendering PDF pages, not preserving them as editable vector scene graphs for arbitrary downstream editing. ³⁹

Preserve vectors with pdf-lib when:

- the imported PDF already corresponds to a page/artboard you can align deterministically,
 - you mostly need to place or clip whole pages or rectangular sections,
 - print fidelity matters and the transform problem is simple.
- pdf-lib can embed whole PDF pages or clipped page regions and draw them back onto a new export document. That is the most credible vector-preserving path available from the browser-side documented APIs in this stack. ⁴⁰

A pragmatic product rule is:

- **PNG/JPG input** → always raster pipeline.
- **PDF input** → offer two modes:
- **Fast mode:** rasterize to a chosen DPI for editing and 3D preview.
- **Print-preserve mode:** keep the original PDF page embedded for export when the placement/masking conditions allow it.

That split is analytically justified by the documented strengths of PDF.js and pdf-lib. For full-SVG preservation, note that pdf-lib's documented high-level SVG entry point is `drawSvgPath()`, not arbitrary whole-SVG document import, so complex SVG preservation is not a turnkey one-call browser workflow in this stack. ⁴¹

Pitfalls and mitigation strategies

Pitfall	Why it happens	Mitigation	Sources
CORS and tainted canvas	Cross-origin images or SVGs drawn without CORS taint the canvas, blocking readback/export and even WebGL texture use	Load user files via <code>File</code> / <code>Blob</code> when possible; for remote assets require CORS; never test with <code>file:///</code> for WebGL textures	42
Large PDFs stall the UI	PDF rendering and image decoding are CPU/memory heavy	Use workers, <code>createImageBitmap</code> , <code>OffscreenCanvas</code> , page-level progressive preview; cap preview DPI while keeping export DPI configurable	43
DPI mismatches	CSS px, raster px, and PDF pt are different units; canvas file metadata defaults can mislead users	Keep mm as source truth; compute raster size explicitly from target DPI; compute PDF size in points; communicate that on-screen size is not a print guarantee	44
Texture bleeding / seams at panel edges	Filtering and mipmapping sample across boundaries; atlas edges and mismatched UVs amplify it	Use exact flat-net UVs, clamp-to-edge where appropriate, add bleed/padding around panel content, and use mipmaps deliberately rather than accidentally	45
Memory leaks after repeated artwork replacement	GPU resources and image bitmaps are not always reclaimed when you expect	Dispose Three.js materials/textures explicitly; close <code>ImageBitmap</code> s; test context-loss scenarios	46

Priority sources for this topic include the CSS Values and Units spec, PDF.js examples/API docs, pdf-lib's API docs for pages/images/SVG/clipping/TrimBox, MDN's canvas export docs, and SVG clipping references. 47

Security, privacy, accessibility, testing, deployment, and implementation plan

Because the app handles user-supplied packaging artwork, privacy and trust matter. A **local-first architecture** is the safest default when no server requirement is specified. Service Workers can support offline-first behavior, IndexedDB can store significant structured data and files/blobs locally, and the File System Access API can provide user-approved save/open flows. Service workers do require HTTPS in deployment, which is an important operational detail. 13

Security-wise, the main browser-specific concerns are not “traditional app servers” so much as file origins, remote-origin images/PDFs, and canvas tainting. PDF.js also documents that URL-based fetching must honor same-origin/CORS rules. If you later add cloud import or asset libraries, make CORS behavior explicit and keep export-critical work off tainted canvases. 42

Accessibility in canvas/WebGL-heavy apps should be treated as **parallel semantic UI**, not as an afterthought. W3C materials on canvas accessibility have long emphasized the challenge that pixels alone are not semantic content. In practical product terms, the authoring app should provide a standard DOM toolbar, native `<button>` elements rather than faux controls, keyboard shortcuts, focus-visible treatment, status text in ARIA live regions, and textual panel/selection summaries outside the canvas/3D scene. For mode switching, a real tablist/tabpanel relationship is better than a visually drawn tab strip inside canvas.

48

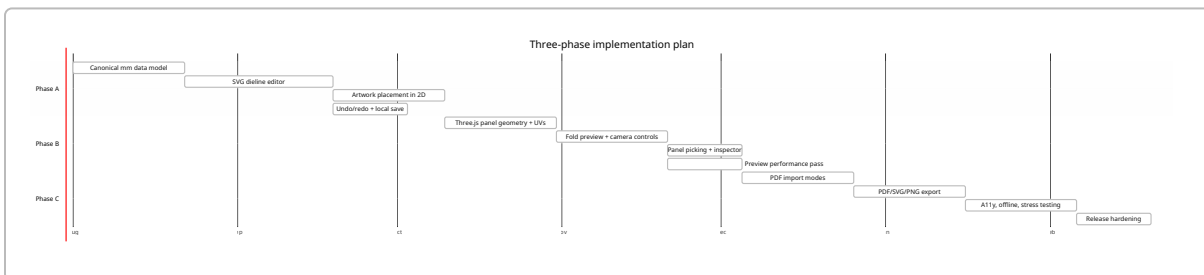
Testing should cover three layers. First, **deterministic math tests**: mm↔pt↔px conversion, panel-net generation, UVs, and fold-angle transforms. Second, **editor interaction tests**: file upload, drag/rotate/scale/nudge, snapping, undo/redo, and mode switching. Third, **visual regression tests**: compare 2D editor output, 3D preview snapshots, and exported PDF/PNG proofs against known-good fixtures. Playwright is strong for browser automation and screenshots/visual comparisons, while Cypress also supports end-to-end, component, and visual-testing workflows. Lighthouse remains useful for performance and accessibility audits even though older PWA-specific audits have shifted over time.

49

For deployment, treat the application as a **PWA-capable browser app**: serve over HTTPS, register a service worker, cache the shell and core dependencies for offline use, and keep project files local unless the user explicitly opts into sync or sharing. If you use WebGPU features, keep a WebGL path available; Three.js' `WebGPURenderer` explicitly supports fallback to a WebGL 2 backend, and Babylon.js similarly documents WebGPU support alongside WebGL compatibility.

50

Three-phase implementation plan



A good delivery strategy is: **Phase A** builds the trustworthy 2D editor and state model; **Phase B** adds the folded preview as a derivation of that model; **Phase C** hardens import/export, accessibility, and offline behavior. That sequencing keeps the most failure-prone, user-visible precision work—structure, placement, units, history—stable before the 3D layer becomes user-facing. This is a design recommendation based on the dependency structure of the stack described above, rather than a claim from any one source. It is also the safest path if later product requirements around flaps, bleed, print marks, collaboration, or vector preservation change.

51

Priority sources for this topic include MDN's Service Worker, Cache, IndexedDB, and File System Access documentation; W3C/MDN accessibility guidance for canvas, native controls, and live regions; Playwright and Cypress test docs; and the Three.js/Babylon renderer documentation for fallback/deployment considerations.

52

1 2 5 15 19 **BufferGeometry - Three.js Docs**

<https://threejs.org/docs/pages/BufferGeometry.html>

3 9 **Core concepts | Docs and Guides**

<https://fabricjs.com/docs/core-concepts/>

4 32 44 47 **CSS Values and Units Module Level 3**

<https://www.w3.org/TR/css-values-3/>

6 14 51 **Implementing Undo History | Redux**

<https://redux.js.org/usage/implementing-undo-history>

7 12 17 21 **- SVG - MDN Web Docs**

https://developer.mozilla.org/en-US/docs/Web/SVG/Reference/Element/clipPath?utm_source=chatgpt.com

8 **Scaling performance - React Three Fiber**

https://r3f.docs.pmnd.rs/advanced/scaling-performance?utm_source=chatgpt.com

10 **WebGPU Support | Babylon.js Documentation**

https://doc.babylonjs.com/setup/support/webGPU?utm_source=chatgpt.com

11 **Graphics**

https://developer.playcanvas.com/user-manual/graphics/?utm_source=chatgpt.com

13 50 52 **Using Service Workers - Web APIs | MDN**

https://developer.mozilla.org/en-US/docs/Web/API/Service_Worker_API/Using_Service_Workers

16 31 **Object3D - Three.js Docs**

<https://threejs.org/docs/pages/Object3D.html>

18 42 **Use cross-origin images in a canvas - HTML | MDN**

https://developer.mozilla.org/en-US/docs/Web/HTML/How_to/CORS_enabled_image

20 35 **Scalable Vector Graphics (SVG) 2**

https://www.w3.org/TR/SVG2/?utm_source=chatgpt.com

22 **ImageBitmap - Web APIs | MDN**

<https://developer.mozilla.org/en-US/docs/Web/API/ImageBitmap>

23 **CanvasTexture - Three.js Docs**

<https://threejs.org/docs/pages/CanvasTexture.html>

24 **OrbitControls - Three.js Docs**

<https://threejs.org/docs/pages/OrbitControls.html>

25 27 **Raycaster - Three.js Docs**

<https://threejs.org/docs/pages/Raycaster.html>

26 **touch-action CSS property - CSS | MDN**

<https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Properties/touch-action>

28 45 **WebGL best practices - Web APIs | MDN**

https://developer.mozilla.org/en-US/docs/Web/API/WebGL_API/WebGL_best_practices

29 **WebGLRenderer - Three.js Docs**

<https://threejs.org/docs/pages/WebGLRenderer.html>

30 KTX2Loader - Three.js Docs

<https://threejs.org/docs/pages/KTX2Loader.html>

33 PDF.js - Examples

<https://mozilla.github.io/pdf.js/examples/>

34 PDF-LIB · Create and modify PDF documents in any JavaScript environment.

<https://pdf-lib.js.org/>

36 HTMLCanvasElement: toBlob() method - Web APIs | MDN

<https://developer.mozilla.org/en-US/docs/Web/API/HTMLCanvasElement/toBlob>

37 40 PDFDocument · PDF-LIB

<https://pdf-lib.js.org/docs/api/classes/pdfdocument>

38 PDFPage · PDF-LIB

<https://pdf-lib.js.org/docs/api/classes/pdfpage>

39 41 JSDoc: Module: pdfjsLib

<https://mozilla.github.io/pdf.js/api/draft/module-pdfjsLib.html>

43 OffscreenCanvas - Web APIs | MDN

<https://developer.mozilla.org/en-US/docs/Web/API/OffscreenCanvas>

46 Material - Three.js Docs

<https://threejs.org/docs/pages/Material.html>

48 Canvas Accessibility - Past, Present and Future

https://www.w3.org/Talks/2014/0510-canvas-a11y/?utm_source=chatgpt.com

49 Actions

https://playwright.dev/docs/input?utm_source=chatgpt.com